GUIDE 03 · START

Connect to anything else via generic JDBC

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=03-connect-via-generic-jdbc. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** a working connection to a database QueryAPIGate has no native driver for - Oracle, SQL Server, DB2, Snowflake, or anything else with a JDBC driver - running saved queries and ad-hoc SQL, with a clear list of what doesn't work on it.

> **Experimental** - may change in any minor release, always noted in the changelog (what that means).

Generic JDBC loads your database vendor's own driver `.jar` into an embedded Java virtual machine. It was verified for this guide with H2's driver standing in for a vendor - the same way the CI tests it - so every command and response below is real. The vendor examples at the end come from each vendor's documentation and weren't run here.

What you'll need

- **Java.** Either the `-h2` Docker image (`ghcr.io/anantharajuc/queryapigate:<version>-h2`), which includes it, or a Java runtime on the host plus `pip install "queryapigate[h2]"` (JayDeBeApi and JPype).
- **Your vendor's JDBC driver** - a `.jar` file from the vendor, readable by QueryAPIGate. In Docker, mount it: `-v /opt/jdbc:/opt/jdbc:ro`.
- From the vendor's documentation: the driver's **class name** and its **JDBC URL** format.

Step 1: Test the details

A `jdbc` connection takes `jar`, `driver_class` and `jdbc_url` instead of `host` / `port` / `database`:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections/test -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "db": "jdbc",
  "jar": "/opt/jdbc/h2-2.2.224.jar",
  "driver_class": "org.h2.Driver",
  "jdbc_url": "jdbc:h2:tcp://localhost:9093/inventory",
  "user": "sa", "password": "${INVENTORY_PASSWORD}"
}'
# {"elapsed_ms": 359.1}
```

The first connection is slow - the JVM starts. `${INVENTORY_PASSWORD}` is read from the server's environment, as for any connection.

Step 2: Save it, and query

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "inventory", "db": "jdbc",
  "jar": "/opt/jdbc/h2-2.2.224.jar", "driver_class": "org.h2.Driver",
  "jdbc_url": "jdbc:h2:tcp://localhost:9093/inventory",
  "user": "sa", "password": "${INVENTORY_PASSWORD}", "active": true
}'

curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"sql": "SELECT id, name, qty FROM parts WHERE qty > :min ORDER BY id", "connection_name":
"inventory", "params": {"min": 0}}'
# [{"ID": 1, "NAME": "bolt", "QTY": 120}, {"ID": 2, "NAME": "nut", "QTY": 300}]
```

Note the column names: they come back as the **driver** reports them - upper case here, because that's H2's habit. Write `AS name` in the SQL if callers should see a particular spelling.

Saved queries work as on any connection - parameters, validation, versions, `?format=`, and streamed exports:

```shell
curl 'http://127.0.0.1:5000/q/parts_in_stock?stream=true&format=csv' -H 'X-API-Key: demo-key'
# ID,NAME,QTY
# 1,bolt,120
# 2,nut,300
```

What doesn't work, verified

FEATURE	ON A `JDBC` CONNECTION
Query time limit	**not enforced.** With `QUERYAPIGATE_QUERY_TIMEOUT=1`, a 2.1-second query ran to the end. JDBC has no portable way to cancel a statement - set a statement timeout in the database itself, or in the JDBC URL if your driver supports one.
Schema browser, `list_tables`	refused: *Schema introspection isn't supported for 'jdbc' connections yet* (`unsupported_operation`). Every vendor's catalogue differs.
`allowed_tables`	a table-restricted key is refused on every query: *Table access restrictions aren't supported for 'jdbc' connections yet* (`table_check_unsupported`) - never left unrestricted.
Read-only, enforced by the database	no - only QueryAPIGate's own SQL check stands between a caller and a write. `DELETE FROM parts` was refused with `read_only`, but give the connection a database user that **can't** write, as the real safeguard.
Table DDL, the tables-and-joins diagram	not available.

The full comparison is in the support matrix.

One JVM per process

All `h2` and `jdbc` connections share one embedded JVM, and its classpath is fixed when it starts - with every jar of every `jdbc` connection configured at that moment. So:

- Configure your connections, then use them: works with no extra steps.
- Add a connection with a **new** jar to a running server: **restart** before using it.

A server with a `jdbc` connection logs at startup that it's experimental: *Experimental: H2 and JDBC connections (connection inventory) …*

Vendor examples (from their documentation - not run here)

DATABASE	DRIVER_CLASS	JDBC_URL
Oracle	`oracle.jdbc.OracleDriver`	`jdbc:oracle:thin:@//db.internal:1521/ORCLPDB1`
SQL Server	`com.microsoft.sqlserver.jdbc.SQLServerDriver`	`jdbc:sqlserver://db.internal:1433;databaseName=sales;encrypt=true`
IBM DB2	`com.ibm.db2.jcc.DB2Driver`	`jdbc:db2://db.internal:50000/SALES`
Snowflake	`net.snowflake.client.jdbc.SnowflakeDriver`	`jdbc:snowflake://<account>.snowflakecomputing.com/?db=SALES&warehouse=WH`

Check the URL format against your driver's version - they change.

Next steps

- Turn your first SQL query into a REST API, on the new connection.
- Set up your first scoped API key.
- Generic JDBC reference.